

Combinatorial optimization layers in machine learning pipelines

ROAPA – ENPC

Guillaume Dalle

gdalle.github.io/

10.11.2025

Contents

Deep learning – setup	2
Deep learning – pipelines	9
Autodiff – basics	15
Autodiff – modes	22
Autodiff – implementation	43
CO layers – theory	51
CO layers – differentiation	60
CO layers – practice	65

Figures without attribution are borrowed from [1] or the works of our team

Deep learning – setup

We are given:

- A set of inputs $\mathcal{X}$ and a set of outputs $\mathcal{Y}$
- With an unknown distribution $p(x, y)$ on $\mathcal{X} \times \mathcal{Y}$

Given a new input x , we want to predict the corresponding output y .

We approximate $y = f_w(x)$ with a parametric model $f_w \in \mathcal{F}$.

Example:

- $\mathcal{F} = \{x \mapsto w^\top x : w \in \mathbb{R}^n\}$ family of linear functions
- $\mathcal{F} = \{x \mapsto \text{NN}_{w(x)} : w \in \mathbb{R}^{\text{big}}\}$ family of neural networks

To measure our prediction error, we have a loss function $\ell(\hat{y})$.

Examples:

- Continuous prediction: $\ell(\hat{y}, y) = \|\hat{y} - y\|^2$
- Discrete prediction: $\ell(\hat{y}, y) = 1_{\hat{y} \neq y}$

Ideally, we want the parameters w that minimize the true risk:

$$\min_w R(w) = \mathbb{E}_{(x,y) \sim p} [\ell(f_w(x), y)]$$

The true distribution p is unknown, but we have training data

$$\mathcal{D}_n = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i \in [1 : n]\}$$

So we minimize the empirical risk instead:

$$\min_w R_n(w) = \frac{1}{n} \sum_{i=1}^n \ell(f_w(x_i), y_i)$$

Amounts to the approximation

$$p \approx \frac{1}{n} \sum_{i=1}^n \delta_{(x_i, y_i)}$$

- Approximation error: the family of functions $\mathcal{F}$ does not include the true data generator
- Optimization error: the empirical risk $R_n(w)$ is not minimized exactly
- Generalization error: the empirical risk $R_n(w)$ is a bad proxy for the true risk $R(w)$

It's hard to minimize all three at the same time!

A good way to define a loss is to adopt a probabilistic framework.

Example: $y = f_w(x) + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ is a Gaussian noise.

Likelihood:
$$p(y \mid x, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-f_w(x))^2}{2\sigma^2}\right)$$

Log-likelihood:
$$\log p(y \mid x, w) = \text{constants} - (y - f_w(x))^2$$

Maximizing log-likelihood = minimizing loss.

Regularization

To avoid overfitting the training set, another term is added to the loss to penalize model complexity.

If we think the weights should remain small, then we solve

$$\min_w R_n(w) + \lambda \|w\|^2$$

Amounts to a Gaussian prior on w that depends on the penalty λ .

Deep learning – pipelines

In deep learning, each scalar is associated with a “neuron” and neurons are structured in layers to propagate information.

- linear layer: $x \in \mathbb{R}^n \mapsto Wx + b \in \mathbb{R}^m$
- activation layer: $x \in \mathbb{R}^n \mapsto \sigma(x) \in \mathbb{R}^n$ (nonlinearities)
- structured layers: convolution, recurrence, message-passing, attention

These layers are chained together from input to output:

$$f_w = f_{w_L}^L \circ f_{w_{L-1}}^{L-1} \circ \dots \circ f_{w_2}^2 \circ f_{w_1}^1$$

where w_ℓ are the weights in layer ℓ .

To minimize the loss, the algorithm of choice is gradient descent:

$$w \leftarrow w - \eta \nabla R_n(w)$$

where the gradient is a sum over samples

$$\nabla R_n(w) = \frac{1}{n} \sum_{i=1}^n \nabla_w [\ell(f_w(x_i), y_i)]$$

Requires:

- Differentiable model f_w
- Differentiable loss ℓ
- Minibatching for large datasets

Logistic regression is a binary classification task defined by the following model:

$$p(y = 1 \mid x, w) = \sigma(w^\top x + b) \quad \text{where } \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Write the empirical risk $R_n(w)$ for a given dataset $\mathcal{D}_n$
- Compute its gradient $\nabla_w R_n(w)$

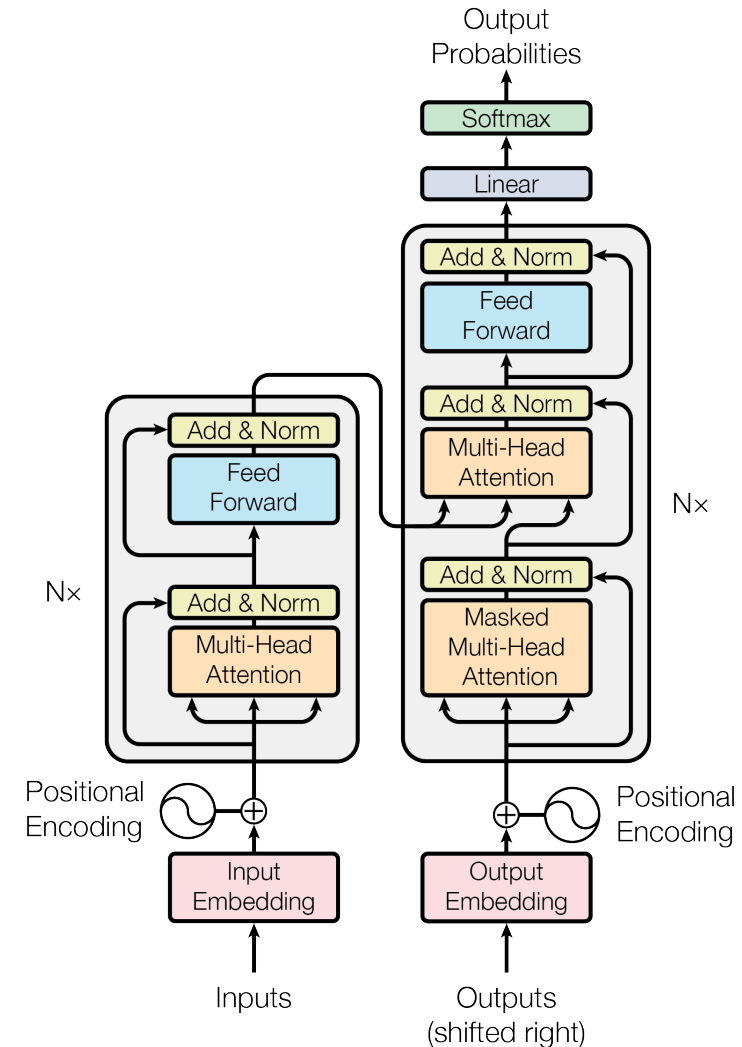
The need for automatic differentiation Deep learning – pipelines

Modern architectures are huge [2].

Don't want manual work when the model changes.

Automatic differentiation enables:

- easy experimentation
- modular code



The big picture

*Differentiable programming is a programming paradigm in which **complex computer programs** (including those with control flows and data structures) can be differentiated end-to-end automatically, enabling gradient-based **optimization of parameters** in the program.*

From the book [1] (see also [3])



Autodiff – basics

Definitions

Derivative = linear approximation of function f around point x :

$$f(x + v) = f(x) + \partial f(x)[v] + o(\|v\|)$$

Here $\partial f(x)[v]$ means “the linear map $\partial f(x)$ applied to v ”.

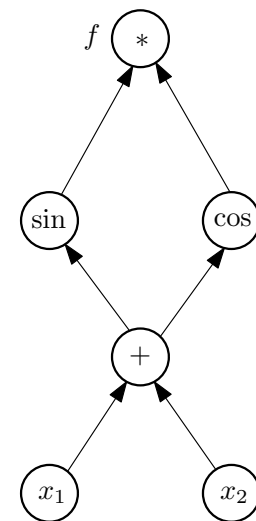
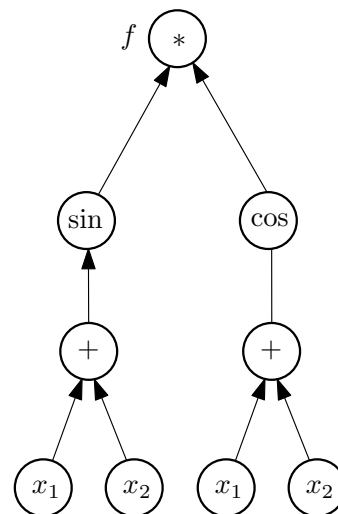
Plug the expression for $f(x)$ into a computer algebra system.

Gives an expression for $f'(x)$, possibly very long.

Expression trees are not great for computer programs:

- shared variables
- loops

Exercise [4]: expression tree & computational graph for $f(x) = \sin(x_1 + x_2) \cos(x_1 + x_2)$



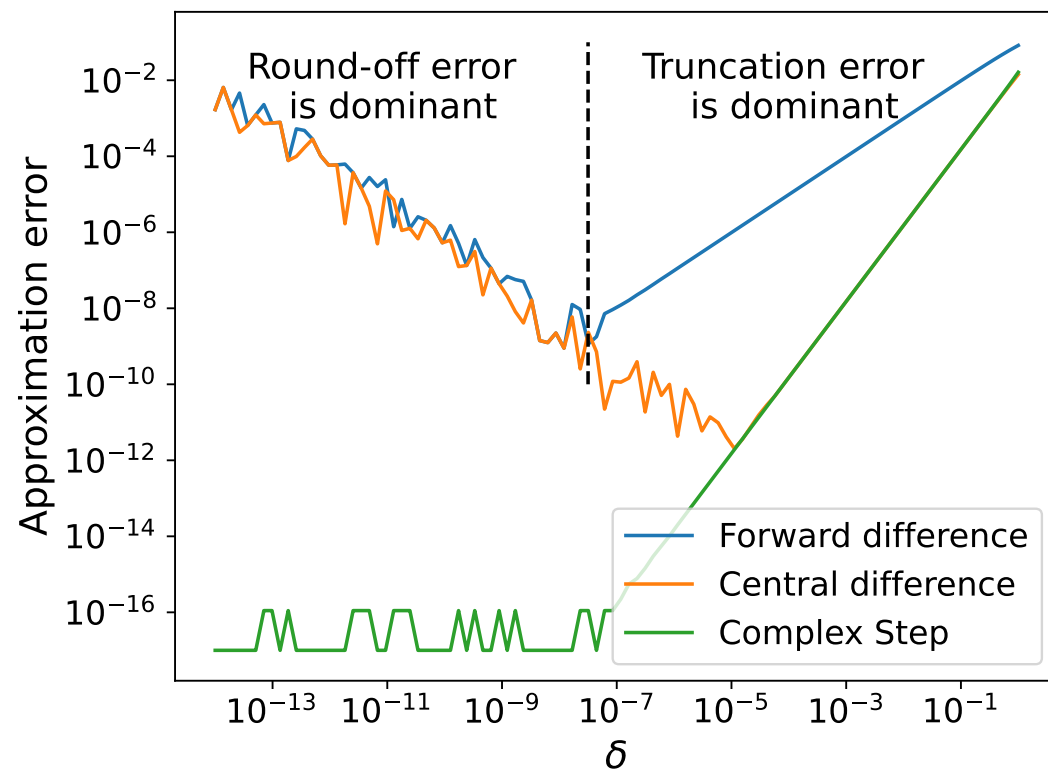
Numeric differentiation

Execute the computational graph f at nearby points (finite differences):

$$\partial f(x)[v] \approx \frac{f(x + \varepsilon v) - f(x)}{\varepsilon}$$

Great at first glance:

- Applies to arbitrary programs
- Only requires two function calls instead of one



Numerical errors:

- Taylor truncation
- Floating-point round-off

Computing a gradient is expensive: $n + 1$ evaluations

$$\nabla f(x) = \begin{pmatrix} \partial_1 f(x) \\ \partial_2 f(x) \\ \dots \\ \partial_n f(x) \end{pmatrix} \approx \frac{1}{\varepsilon} \begin{pmatrix} f(x + \varepsilon e_1) - f(x) \\ f(x + \varepsilon e_2) - f(x) \\ \vdots \\ f(x + \varepsilon e_n) - f(x) \end{pmatrix}$$

One perturbation per input dimension!

Transform the computational graph f into a new graph ∂f and propagate derivatives through it:

- Keeps the compact program encoding ($\neq$ symbolic)
- Yields exact derivative values ($\neq$ numeric)
- Can compute gradients efficiently (in reverse mode)

Automatic differentiation = chain rule + layers with known derivatives.

Autodiff – modes

Derivatives as linear maps

If $f : \mathbb{R}^n \longrightarrow \mathbb{R}^m$, then $\partial f(x)$ can be represented as a Jacobian matrix:

$$J_f(x) = \left(\frac{\partial f_i}{\partial x_j}(x) \right)_{i,j} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{pmatrix}$$

However, the linear map $v \longmapsto \partial f(x)[v]$ is natural to work with:

- generalizes to arbitrary vector spaces
- no need to materialize a matrix or flatten anything

The chain rule

Given a function composition $f = h \circ g$ with two layers, we have

$$\partial f(x) = \partial h(g(x)) \circ \partial g(x)$$

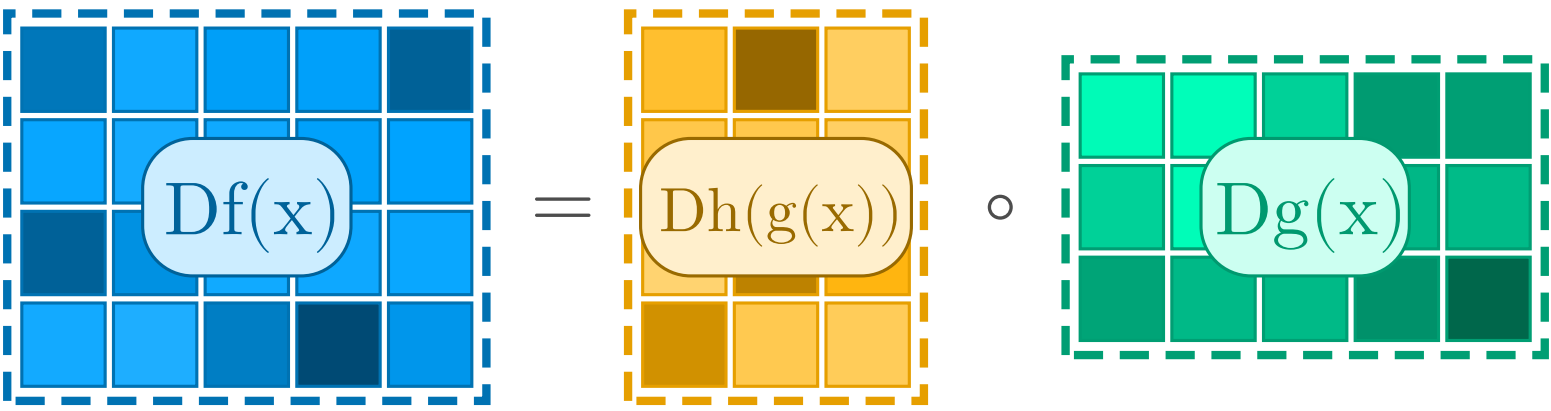
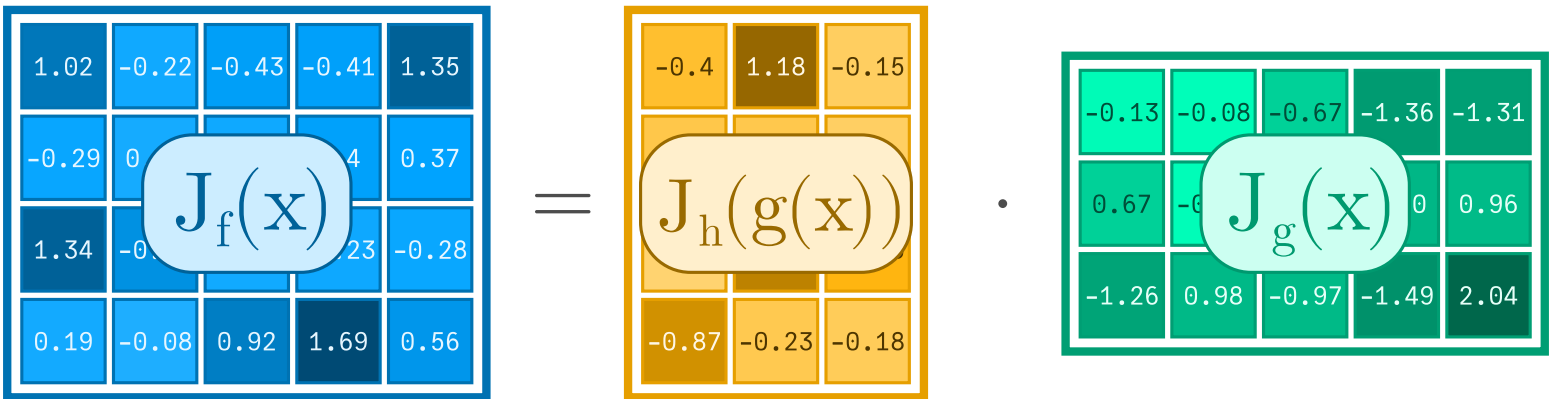
The derivative of f is the composition of two linear maps:

$$\partial f(x) : u \xrightarrow{\partial g(x)} v \xrightarrow{\partial h(g(x))} w$$

We can differentiate any function knowing the derivatives of its layers.

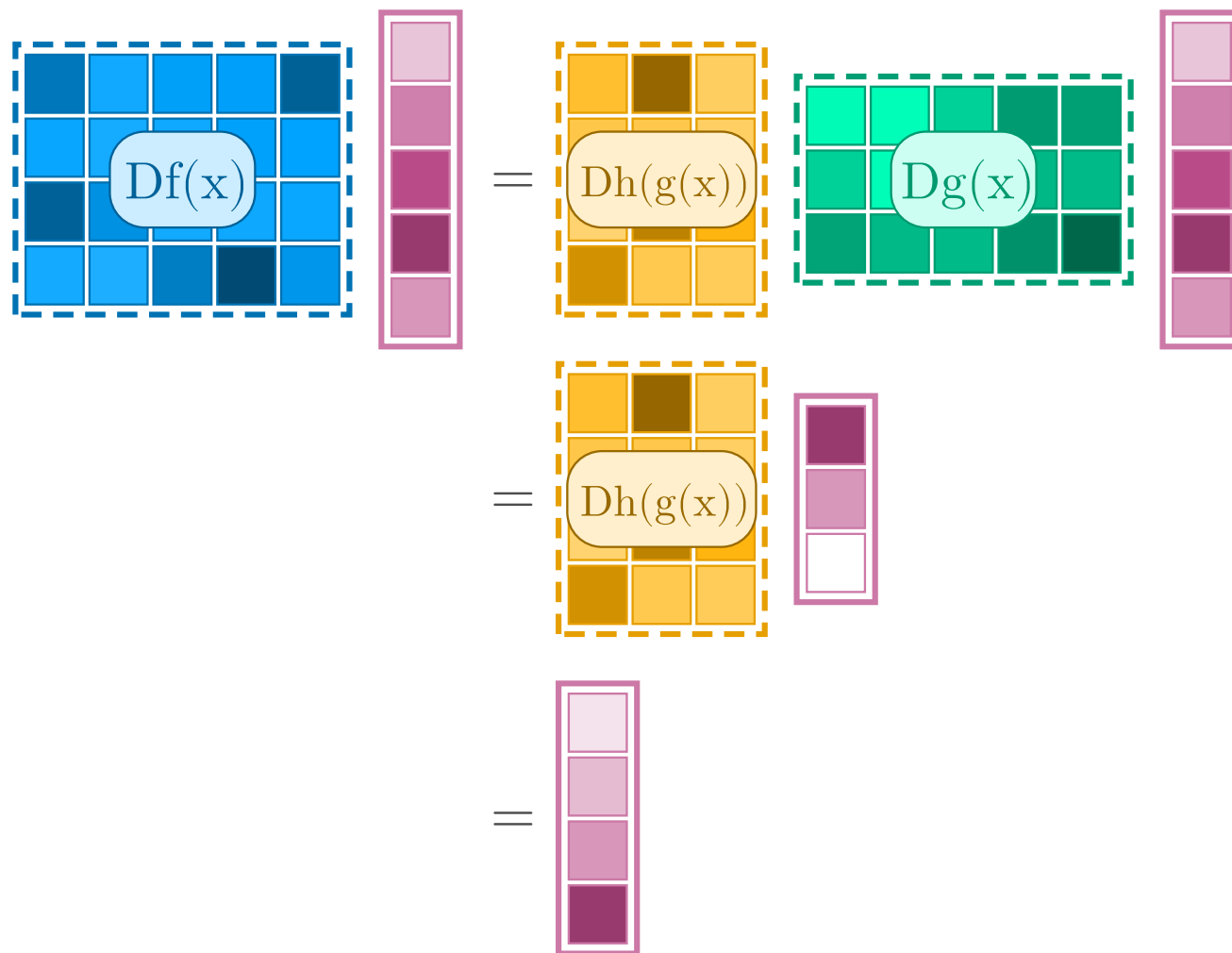
From matmul to map composition

Autodiff – modes



From matmul to map composition (2)

Autodiff – modes



Scalar layers

Let $\dot{x}$ denote an arbitrary input tangent (directional derivative).

variables	function f	derivative $\partial f(x)[\dot{x}]$
$x \in \mathbb{R}$	ax	$a\dot{x}$
$x \in \mathbb{R}$	$\sin(x)$	$\cos(x)\dot{x}$
$x, y \in \mathbb{R}$	x, y	$x\dot{y} + y\dot{x}$

This mirrors exactly what we learned in high school.

Array layers

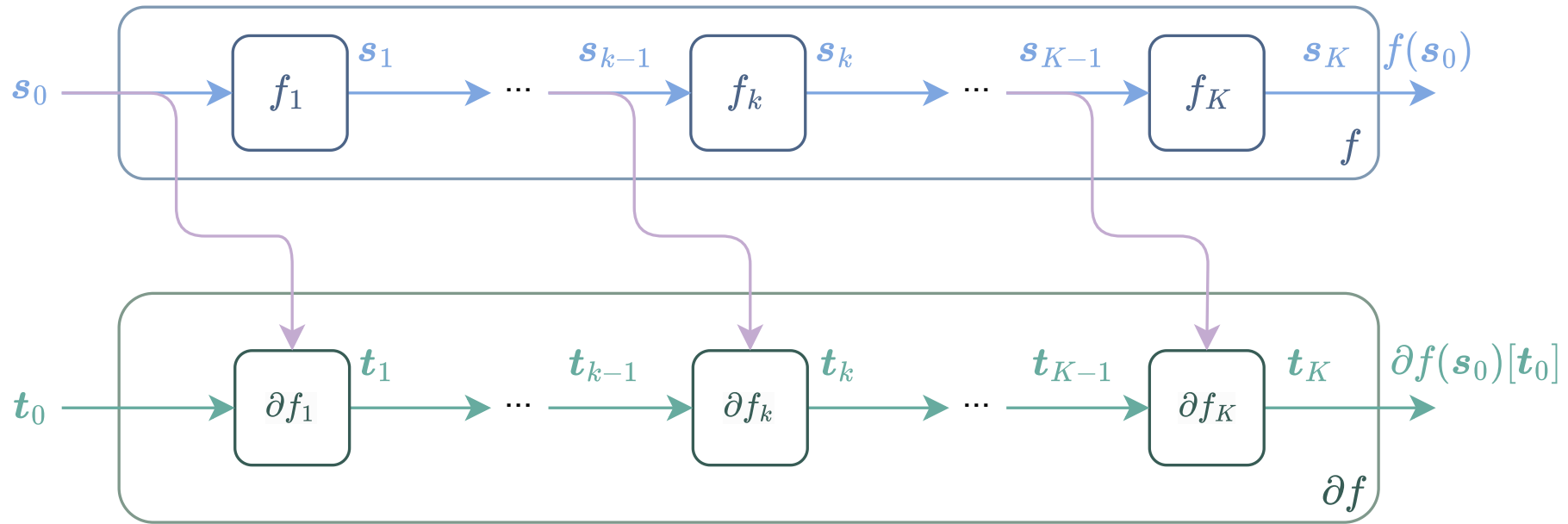
Define rules for array functions with known derivatives [5].

variables	function $f(x)$	derivative $\partial f(x)[\dot{x}]$
$x \in \mathbb{R}^n$	$Ax + b$	$A\dot{x}$
$x \in \mathbb{R}^n$	$\sigma(x)$	$\sigma'(x)\dot{x}$
$x \in \mathbb{R}^n$	$\ x\ ^2$	$2x^\top \dot{x}$
$X \in \mathbb{R}^{n \times n}$	X^{-1}	$-X^{-1} \dot{X} X^{-1}$

For the inverse, a Jacobian matrix of size $(n \times n)^2$ is not needed: the linear map is more efficient.

This was forward mode

Propagate the input and its tangent together through a chain of layers.



Back to matrices, forward mode computes JVPs $\partial f(x)[v] = J_{f(x)}v$.

With $v = e_j$, this gives a column of the Jacobian matrix:

$$\partial f(x)[e_j] = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(x) & \dots & \frac{\partial f_1}{\partial x_j}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \frac{\partial f_2}{\partial x_1}(x) & \dots & \frac{\partial f_2}{\partial x_j}(x) & \dots & \frac{\partial f_2}{\partial x_n}(x) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \dots & \frac{\partial f_m}{\partial x_j}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{pmatrix}$$

A tale of columns and rows

For a scalar-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the Jacobian has one row:

$$J_f(x) = \nabla f(x)^\top = \left(\frac{\partial f}{\partial x_1}(x) \quad \frac{\partial f}{\partial x_2}(x) \quad \dots \quad \frac{\partial f}{\partial x_n}(x) \right)$$

We need n forward-mode JVPs to compute it column by column:

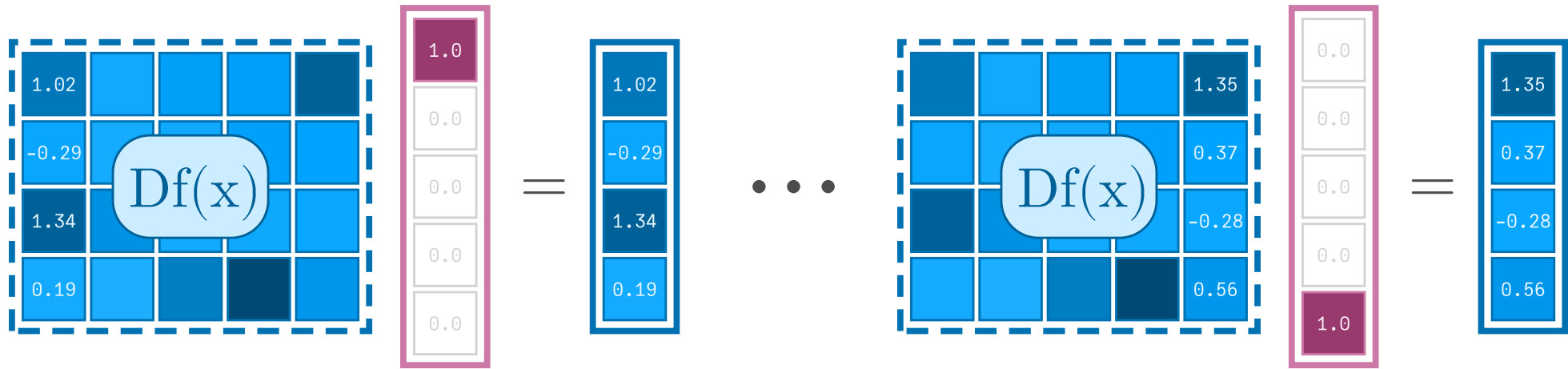
$$\partial f(x)[e_1] = \left(\frac{\partial f}{\partial x_1}(x) \quad \frac{\partial f}{\partial x_2}(x) \quad \dots \quad \frac{\partial f}{\partial x_n}(x) \right)$$

$$\partial f(x)[e_2] = \left(\frac{\partial f}{\partial x_1}(x) \quad \frac{\partial f}{\partial x_2}(x) \quad \dots \quad \frac{\partial f}{\partial x_n}(x) \right)$$

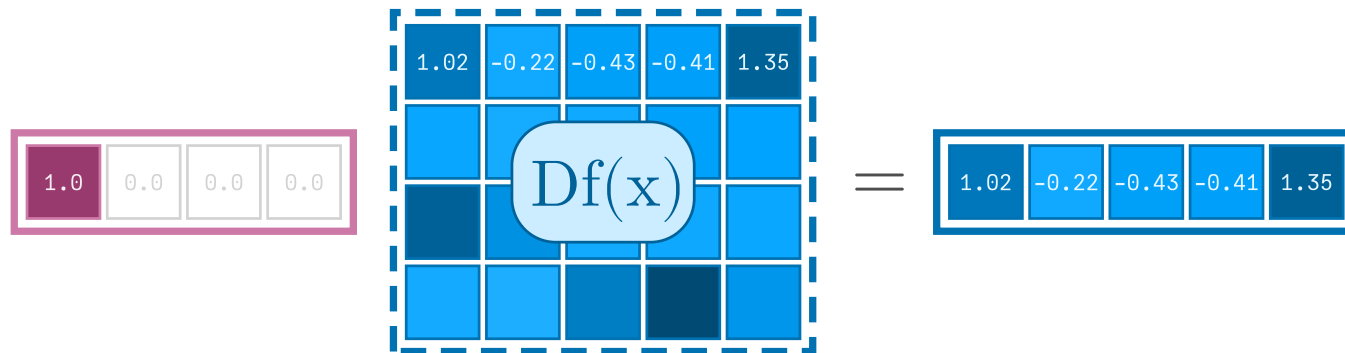
Can we compute it row by row instead, in one shot?

Columnwise Jacobian

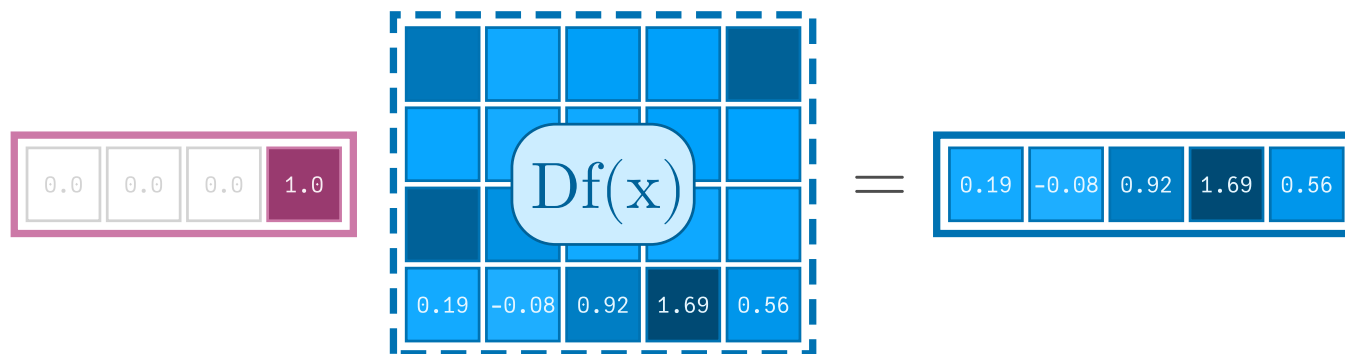
Autodiff – modes



Bad for gradients (just one row)



...



Good for gradients
(just one row)

Transpositions and adjoints

The adjoint of a linear map $\ell : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is the only linear map $\ell^* : \mathbb{R}^m \rightarrow \mathbb{R}^n$ such that

$$\forall (x, y) \in \mathbb{R}^n \times \mathbb{R}^m, \quad \langle \ell(x), y \rangle = \langle x, \ell^*(y) \rangle$$

If ℓ is represented by a matrix A , then ℓ^* is represented by $A^\top$.

If we could compute $\partial f(x)^*$, it would give us rows of the Jacobian through VJPs:

$$\partial f(x)^*[e_i] = J_{f(x)}^\top e_i = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(x) & \dots & \frac{\partial f_1}{\partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_i}{\partial x_1}(x) & \dots & \frac{\partial f_i}{\partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(x) & \dots & \frac{\partial f_m}{\partial x_n}(x) \end{pmatrix}$$

In particular, the gradient is just $\nabla f(x) = \partial f(x)^*[1]$.

Adjoint chain rule

Adjoint of linear maps reverse order, like matrix transposes:

$$\begin{aligned}\partial f(x) &= \partial h(g(x)) \circ \partial g(x) \\ \partial f(x)^* &= \partial g(x)^* \circ \partial h(g(x))^*\end{aligned}$$

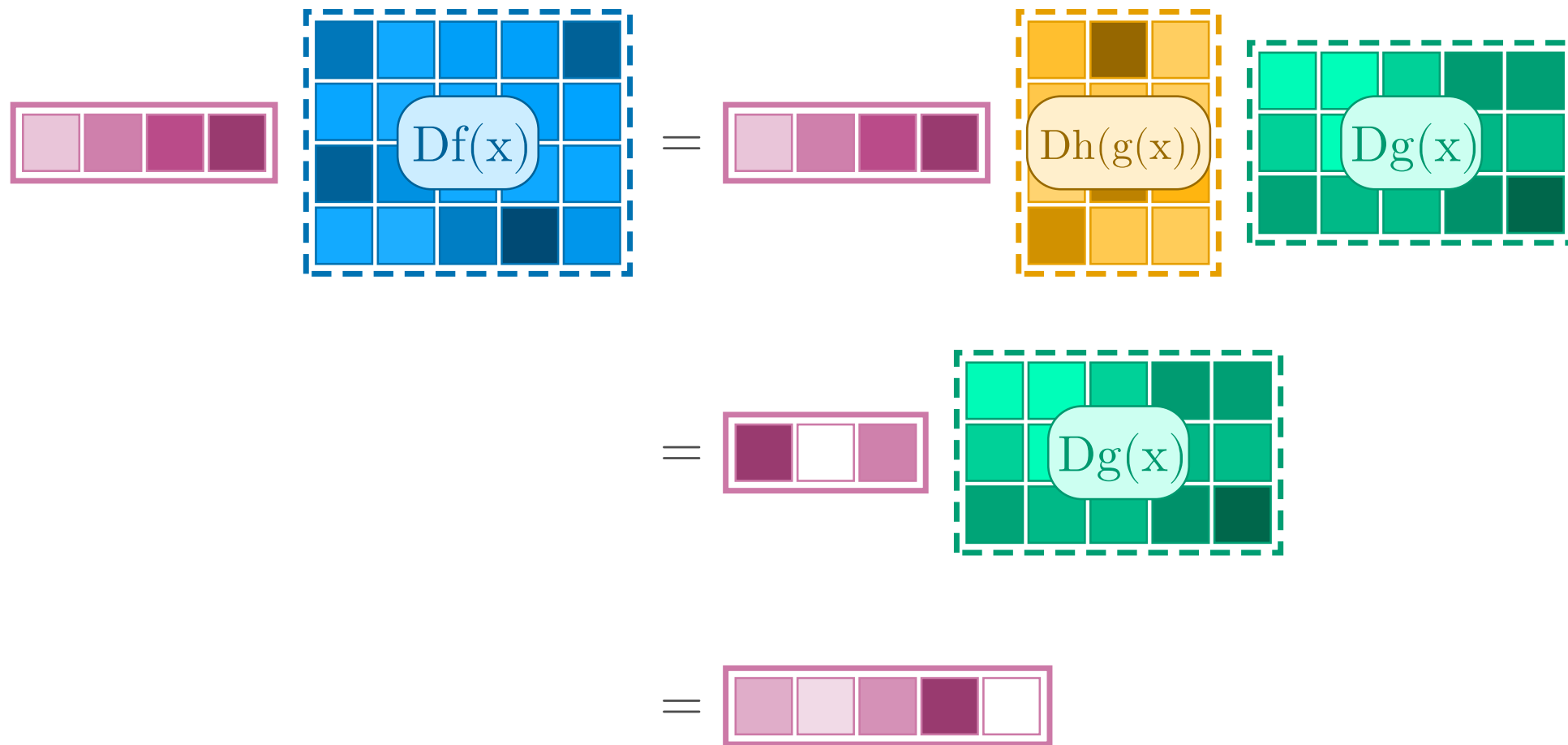
Now the propagation happens from the output back to the input:

$$\partial f(x)^* : u \xleftarrow{\partial g(x)^*} v \xleftarrow{\partial h(g(x))^*} w$$

We can differentiate any function in reverse mode knowing the adjoint derivatives of its layers.

Transposed matmul & adjoint composition

Autodiff – modes



Back to our layer examples

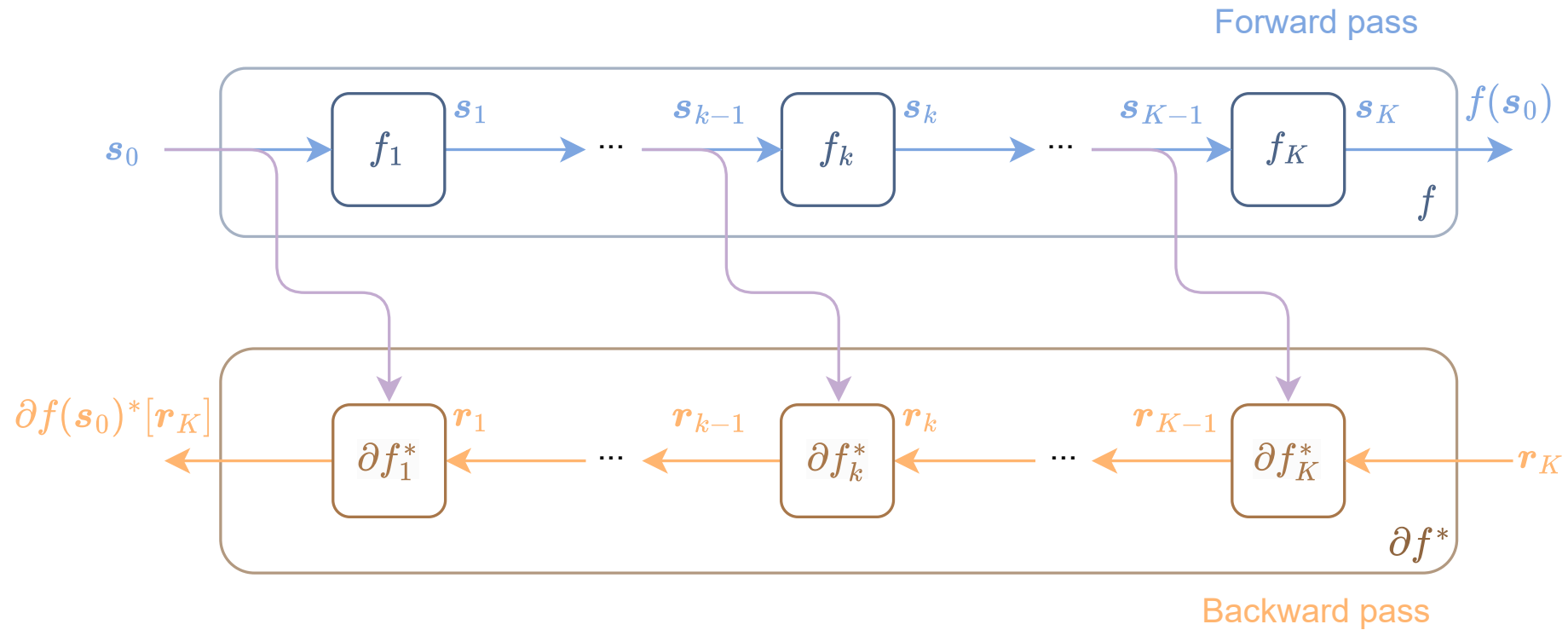
Let $\bar{y}$ be an arbitrary output cotangent (sensitivity).

variables	output	function $f(x)$	adjoint $\partial f(x)^*[\bar{y}]$
$x \in \mathbb{R}$	$y \in \mathbb{R}$	$\sin(x)$	$\cos(x)\bar{y}$
$x \in \mathbb{R}^n$	$y \in \mathbb{R}^m$	$Ax + b$	$A^\top \bar{y}$
$x \in \mathbb{R}^n$	$y \in \mathbb{R}$	$\ x\ ^2$	$2x\bar{y}$
$X \in \mathbb{R}^{n \times n}$	$Y \in \mathbb{R}^{n \times n}$	X^{-1}	$-X^{-\top} \bar{Y} X^{-\top}$

More crazy formulas in [6].

This was reverse mode

Propagate the input through a chain of layers, record enough information, backpropagate the output cotangent.



Time complexity

Theorem (Baur-Strassen):

- Cost of 1 JVP (forward mode) $\propto$ cost of 1 function call
- Cost of 1 VJP (reverse mode) $\propto$ cost of 1 function call

Rather easy to believe:

- Individual derivatives not much harder than the corresponding layer
- Composition of linear maps adds up their computational costs

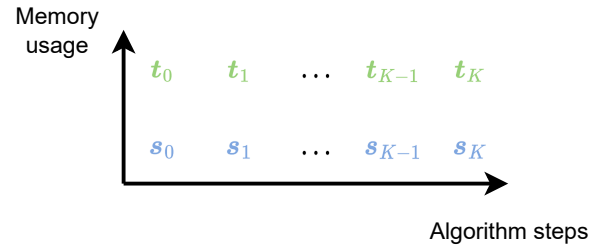
Time complexity (special cases)

Assume the function f can be computed in time $O(\tau)$.

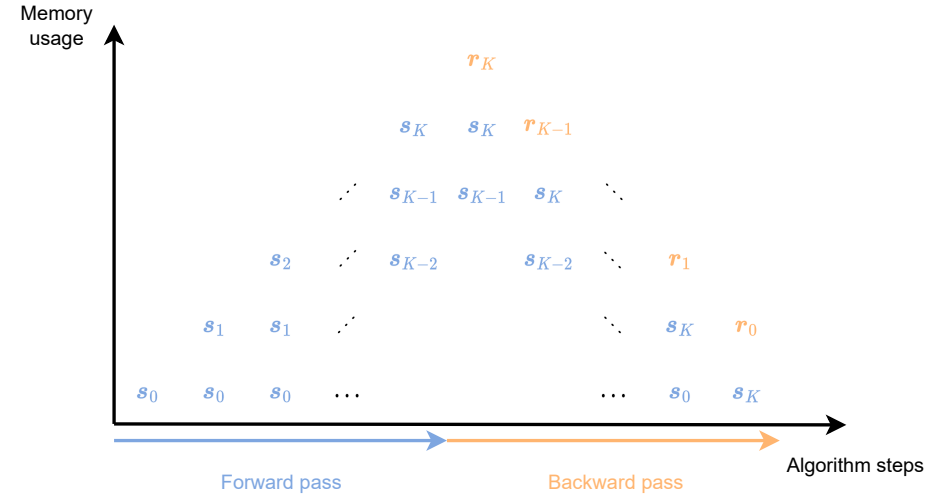
What is the complexity of computing a full Jacobian matrix?

setting	object	forward mode	reverse mode
$\mathbb{R}^n \rightarrow \mathbb{R}^m$	Jacobian matrix	$O(n\tau)$	$O(m\tau)$
$\mathbb{R}^n \rightarrow \mathbb{R}$	gradient vector	$O(n\tau)$	$O(\tau)$
$\mathbb{R} \rightarrow \mathbb{R}^m$	derivative vector	$O(\tau)$	$O(m\tau)$

Space complexity



Forward mode has constant memory cost.



Reverse mode has linear memory cost (in the depth of the chain).

Autodiff – implementation

Autodiff reinterprets a computer program to mean something else [7].

We can take the same idea in other directions:

- Uncertainty propagation
- Physical unit checking
- Sparsity detection
- Kernel fusion

Operator overloading

Pass augmented values to language operators & overload their behavior.

Example: Dual numbers in ForwardDiff.jl [8].

```
using ForwardDiff: Dual
```

```
u, ð = 2.0, 3.0
```

```
v, v̇ = 4.0, 5.0
```

```
du = Dual(u, ð)
```

```
dv = Dual(v, v̇)
```

```
julia> du * dv  
Dual{Nothing}(8.0, 22.0)
```

```
julia> u * v̇ + v * ð  
22.0
```

```
julia> du / dv  
Dual{Nothing}(0.5, 0.125)
```

```
julia> (ð * v - v̇ * u) / v^2  
0.125
```

Preprocess the source code to add derivative bookkeeping.

Example: jaxpr intermediate representation in JAX [9]

```
import jax
import jax.numpy as jnp

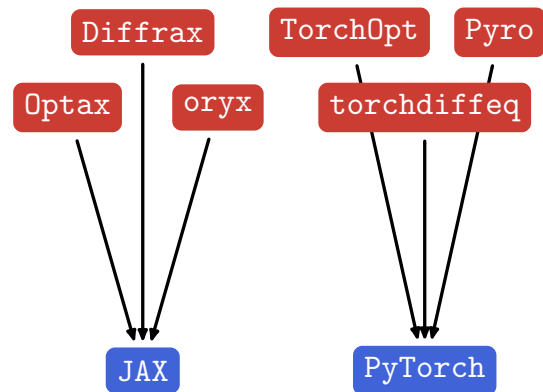
def selu(x, alpha=1.67, lambda_=1.05):
    return lambda_ * jnp.where(
        x > 0,
        x,
        alpha * jnp.exp(x) - alpha
    )

x = jnp.arange(5.0)
jax.make_jaxpr(selu)(x)
```

```
{ lambda ; a:f32[5]. let
  b:bool[5] = gt a 0.0:f32[]
  c:f32[5] = exp a
  d:f32[5] = mul 1.6699999570846558:f32[] c
  e:f32[5] = sub d 1.6699999570846558:f32[]
  f:f32[5] = jit[
    name=_where
    jaxpr={ lambda ; b:bool[5] a:f32[5]
e:f32[5]. let
  f:f32[5] = select_n b e a
  in (f,) }
] b a e
  g:f32[5] = mul 1.0499999523162842:f32[] f
  in (g,) }
```

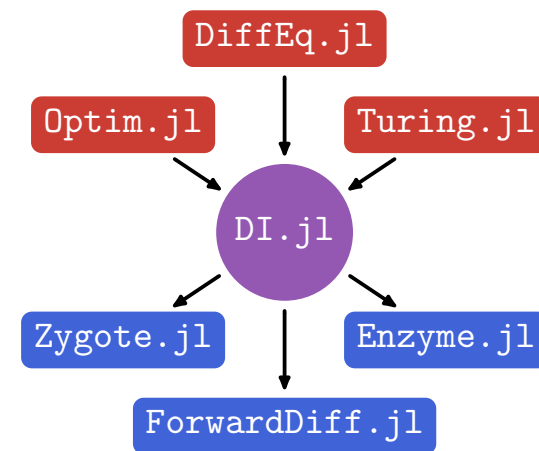
Python libraries

- [autograd](#) (historical)
- [TensorFlow](#)
- [PyTorch](#)
- [JAX](#)



Julia libraries (see [10])

- [ForwardDiff.jl](#)
- [Zygote.jl](#)
- [Enzyme.jl](#)
- [Mooncake.jl](#)



A subset of the language

Autodiff – implementation

Not every framework can differentiate every operation.

- PyTorch, JAX: restricted to domain-specific sublanguage
- Zygote.jl, JAX: error on code with mutation
- ForwardDiff.jl: errors on code with type constraints



[Open in Colab](#) [Open in Kaggle](#)

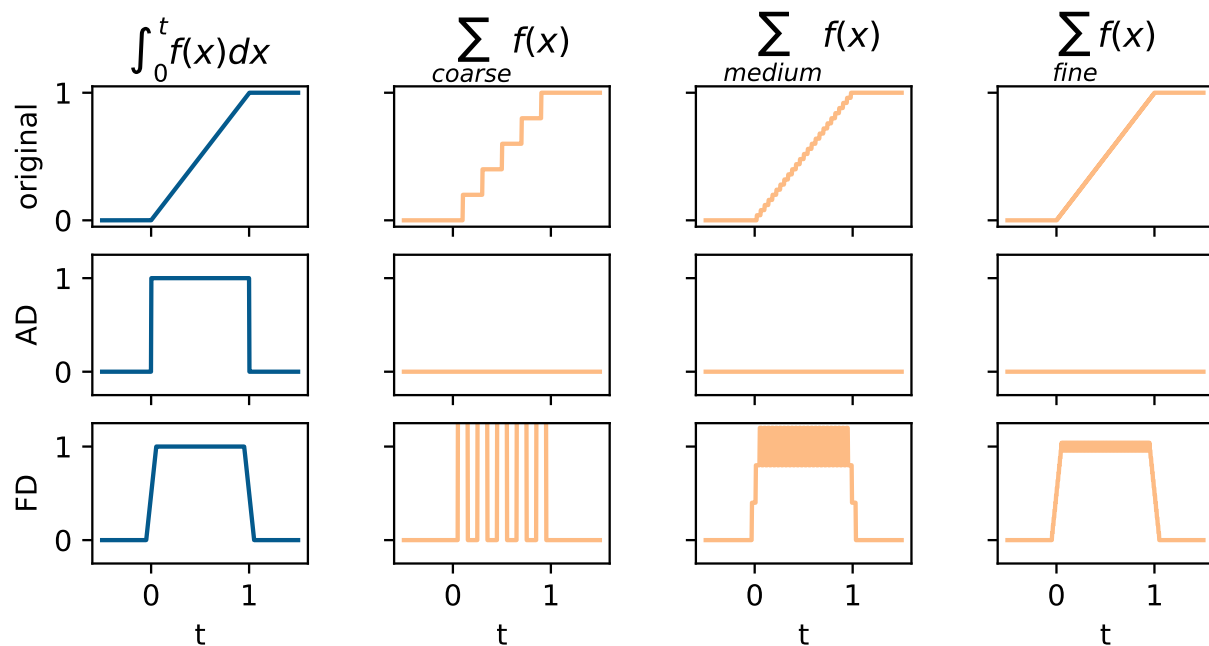
When walking about the countryside of Italy, the people will not hesitate to tell you that **JAX** has *"[una anima di pura programmazione funzionale](#)"*.

JAX is a language for **expressing** and **composing transformations** of numerical programs. **JAX** is also able to **compile** numerical programs for CPU or accelerators (GPU/TPU). JAX works great for many numerical and scientific programs, but **only if they are written with certain constraints** that we describe below.

When to stop autodiffing

A function is approximated by a program [11]. We can either

1. Differentiate the approximation
2. Approximate the derivative



Example: Heron's method

Autodiff – implementation

The following iteration approximates the square root of $a \in [0, \infty[$:

$$x_0 = a, \quad x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$$

$$x_0 = a$$

$$\dot{x}_0 = 1$$

While not converged

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$$

$$\dot{x}_{n+1} = \frac{1}{2} \left(\dot{x}_n + \frac{1}{x_n} - \frac{a \dot{x}_n}{x_n^2} \right)$$

Return $\dot{x}_N$

$$x_0 = a$$

While not converged

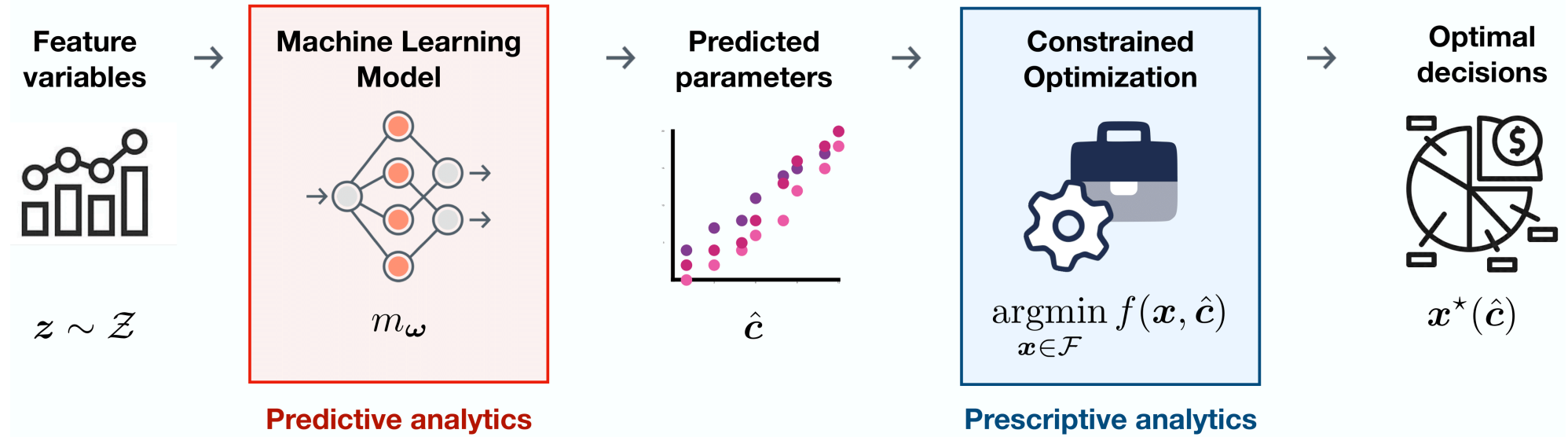
$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$$

Return $\frac{1}{2x_N}$

CO layers – theory

Decision-focused learning

Using operations research solvers inside learning pipelines [12].



- Ensures constraint satisfaction (better ML)
- Makes a solver data-driven (better CO)

Notations

Typical pipeline focuses on learning costs, not constraints:

- x : input (contains the instance data)
- θ : cost vector
- y : solution to the optimization problem

Machine learning model: $\hat{\theta} = f_w(x)$

Optimization solver (linear objective):

$$\hat{y} = \operatorname{argmax} \left\{ \hat{\theta}^\top y : y \in \mathcal{Y}(x) \right\}$$

where $\mathcal{Y}(x)$ contains feasible solutions for x

Prediction-focused learning

- Dataset: inputs and cost vectors $\{(x_i, \bar{\theta}_i)\}$
- Objective: minimize the error on cost prediction alone

$$\min_w \frac{1}{n} \sum_{i=1}^n \ell(\hat{\theta}_i, \bar{\theta}_i) \quad \text{with} \quad \hat{\theta}_i = f_w(x_i)$$

Problem: doesn't take downstream decision into account, treats all errors the same.

Idea explained by [13]: learn to predict a cost *such that the downstream decision is good*.

True cost of the downstream decision: $\bar{\theta}^\top \hat{y}$

$$\min_w \frac{1}{n} \sum_{i=1}^n \bar{\theta}_i^\top \left(\operatorname{argmin}_{y_i \in \mathcal{Y}(x_i)} \hat{\theta}_i^\top y_i \right) \quad \text{with} \quad \hat{\theta}_i = f_w(x_i)$$

Learning by solution imitation

Sometimes, true costs are not known. Then, we learn to imitate known solutions, with a decision-aware loss:

$$\min_w \frac{1}{n} \sum_{i=1}^n \ell(\hat{\theta}_i, y_i) \quad \text{with} \quad \hat{\theta}_i = f_w(x_i)$$

Learning by experience

Most of the times, optimal solutions for training are expensive.

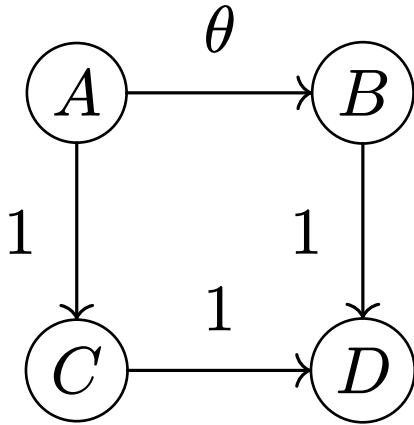
Then we just use a black-box cost evaluator:

$$\min_w \frac{1}{n} \sum_{i=1}^n c_{x_i} \left(\operatorname{argmin}_{y_i \in \mathcal{Y}(x_i)} \hat{\theta}_i^\top y_i \right) \quad \text{with} \quad \hat{\theta}_i = f_w(x_i)$$

Similar to reinforcement learning but allows more structured actions.

Differentiation challenge

Discrete solvers and program elements don't have useful derivatives.



$$\text{shortest_path}(\theta) = \begin{cases} \text{ABD} & \text{if } \theta < 1 \\ \text{ACD} & \text{if } \theta > 1 \\ \text{both} & \text{if } \theta = 1 \end{cases}$$

Piecewise-constant function!

We need a nicely differentiable surrogate to allow backpropagation.

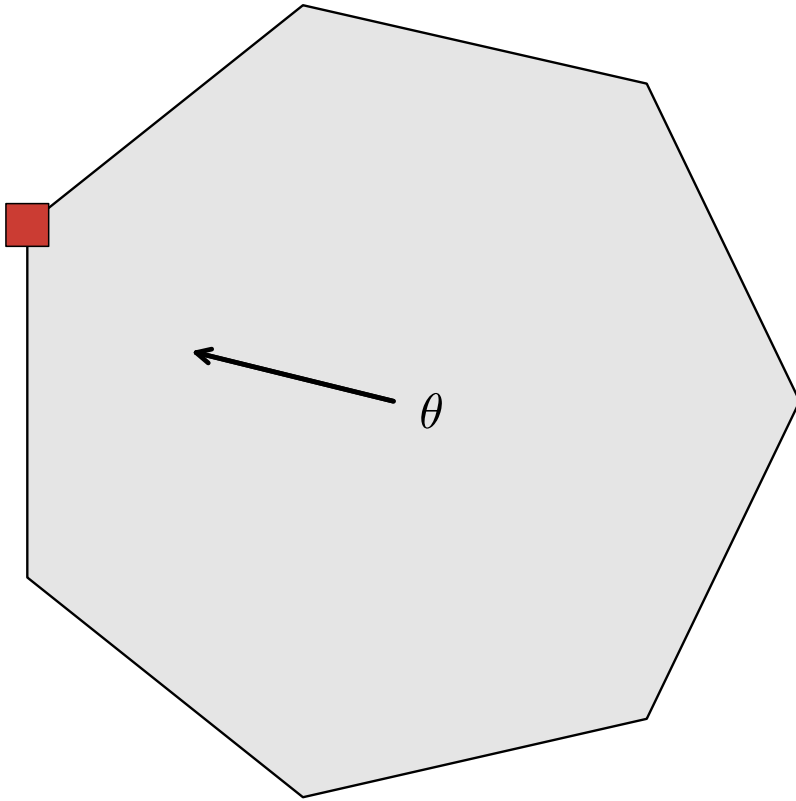
Linear programs

$$y(\theta) = \operatorname{argmax} \{ \theta^\top y : Ay \leq b \}$$

Feasible set is a polyhedron.

Almost surely on θ , the optimum is a vertex.

Requires smoothing!



CO layers – differentiation

Three paradigms:

- Regularization
- Integration
- Clever loss

Regularization

Solve a convex program instead!

Quadratic regularization [14]:

$$y_\gamma(\theta) = \operatorname{argmax}_y \theta^\top y - \gamma \|y\|^2 \quad \text{s.t.} \quad Ay \leq b$$

Logarithmic barrier [15]:

$$y_\gamma(\theta) = \operatorname{argmax}_y \theta^\top y - \gamma \sum_i \log(s_i) \quad \text{s.t.} \quad Ay + s = b$$

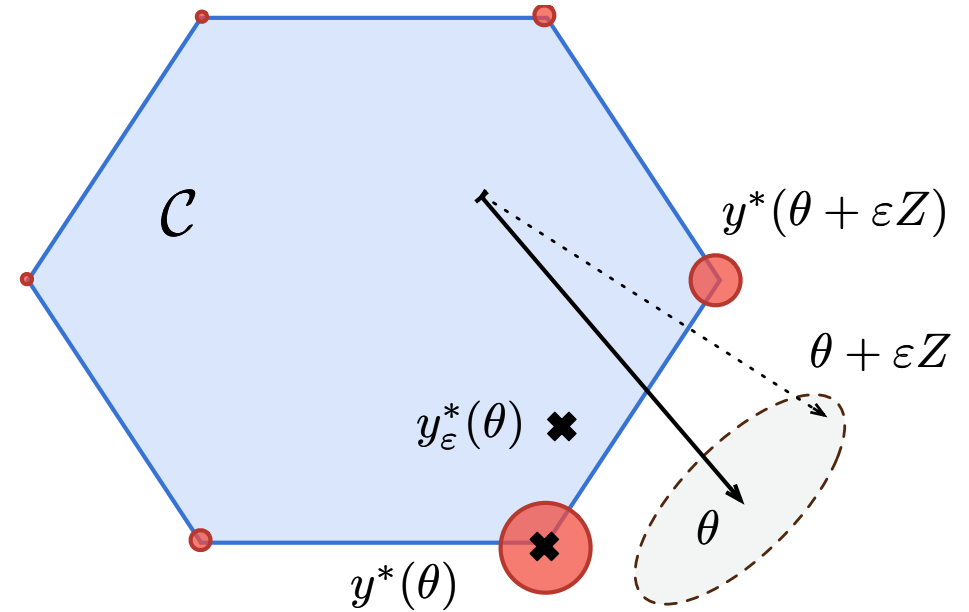
Must use a different solver and differentiate the convex problem.

Integration

Instead of picking a single polytope vertex, construct a probability distribution $p_\theta(y)$ over solutions [16], [17], [18].

Replace discrete optimizer with an expectation.

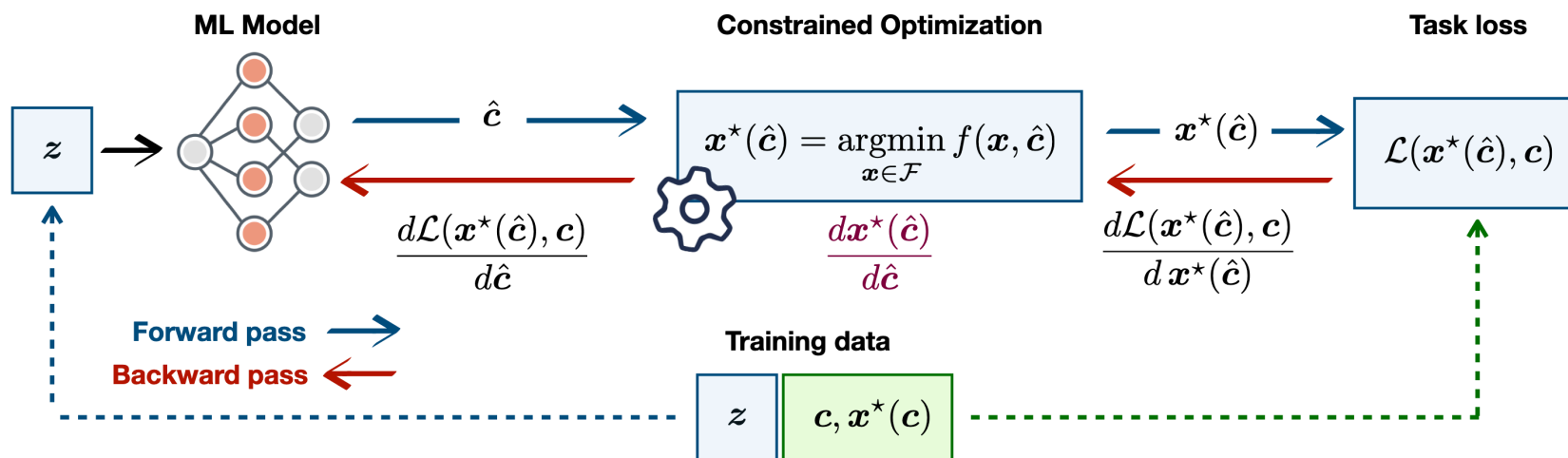
$$y_\varepsilon(\theta) = \mathbb{E} \left[\operatorname{argmax}_{y \in \mathcal{Y}} (\theta + \varepsilon Z)^\top y \right]$$



Differentiating through the solver itself may not be necessary.

Some loss functions provide (sometimes convex) surrogates [12]:

- SPO+ loss [13]
- Fenchel-Young loss [19]



CO layers – practice

Key design choices in decision-focused learning:

- Choice of optimization solver
 - Practical relevance (close to real problem)
 - Fast enough (to be used in training)
- Choice of machine learning model
 - Expressivity (capture relevant features)
 - Variable size (adapts to small and large instances)
 - Invariance properties (respect problem symmetries)

Many problems are on graphs $x = (\mathcal{G}, \mathcal{E})$, with a cost function per edge:

$$y(\theta) = \operatorname{argmin}_y \sum_{e \in \mathcal{E}} \theta_e y_e \quad \text{s.t.} \quad y \in \mathcal{Y}$$

Very simple model: combines edge features φ_e with same weights w

$$\theta_e = \sigma(w^\top \varphi_e(x))$$

No interactions between edges in the model.

Graph Neural Networks

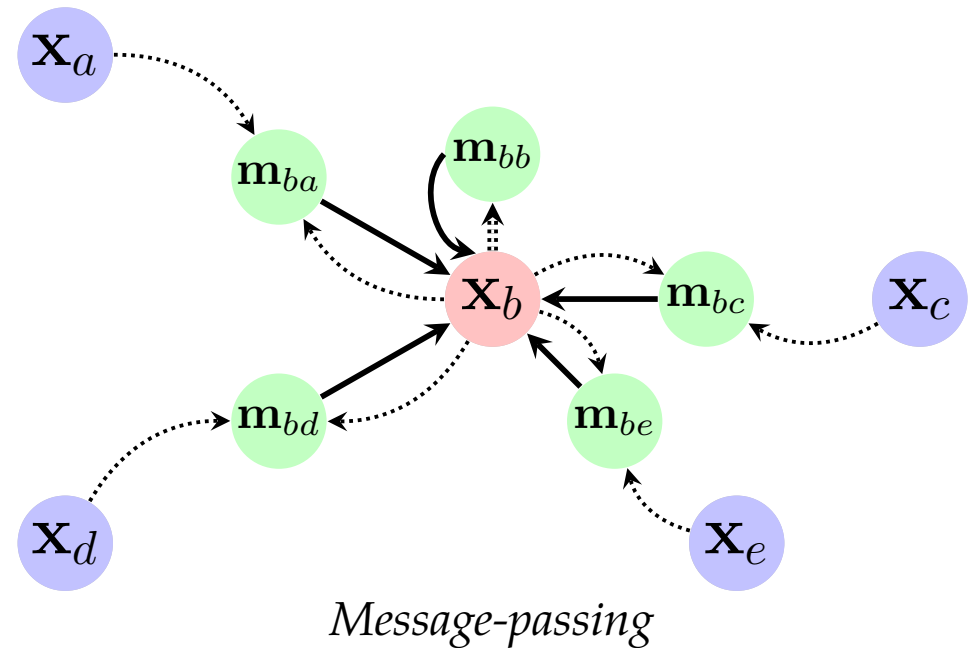
Graph-equivariant functions f are defined on node features X such that

$$f(PX) = Pf(X) \text{ for all permutations } P$$

Message-passing neural networks [20] are one such family:

$$h_u = \varphi \left(x_u, \bigoplus_{v \in \mathcal{N}_u} \psi(x_u, x_v) \right)$$

which aggregates information over neighborhoods

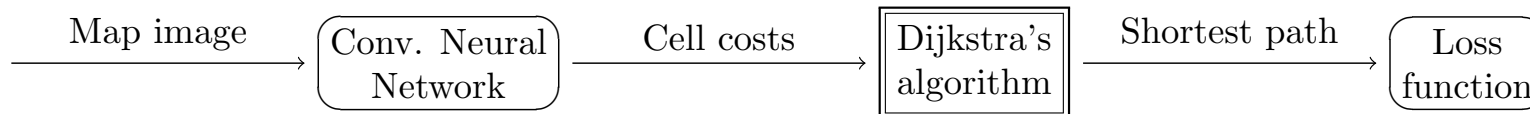
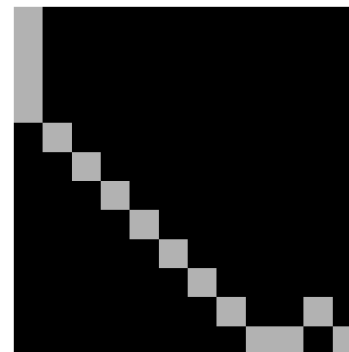


For each of the problems that follow:

- Choose an appropriate optimization solver
- Choose an appropriate statistical model
- Suggest relevant features
- Imagine a practical situation corresponding to each learning scenario

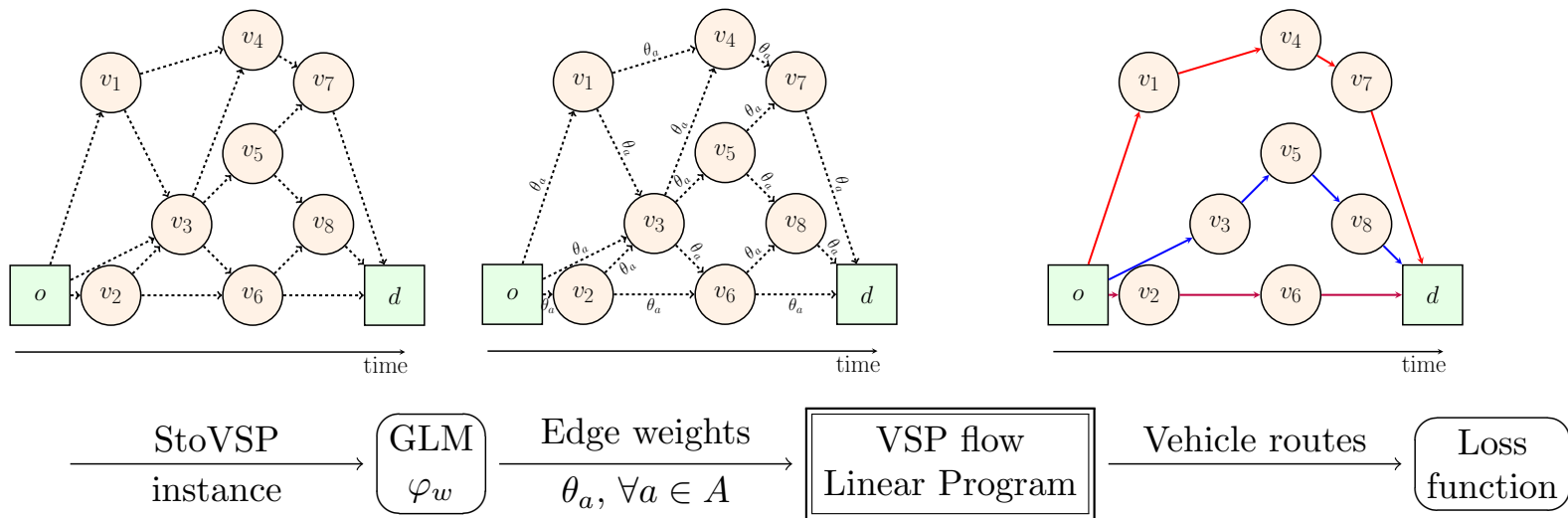
Exercise: shortest paths on a map

- You are: an explorer
- I give you: a satellite image of a country
- You give me: the shortest path between the northernmost and southernmost points



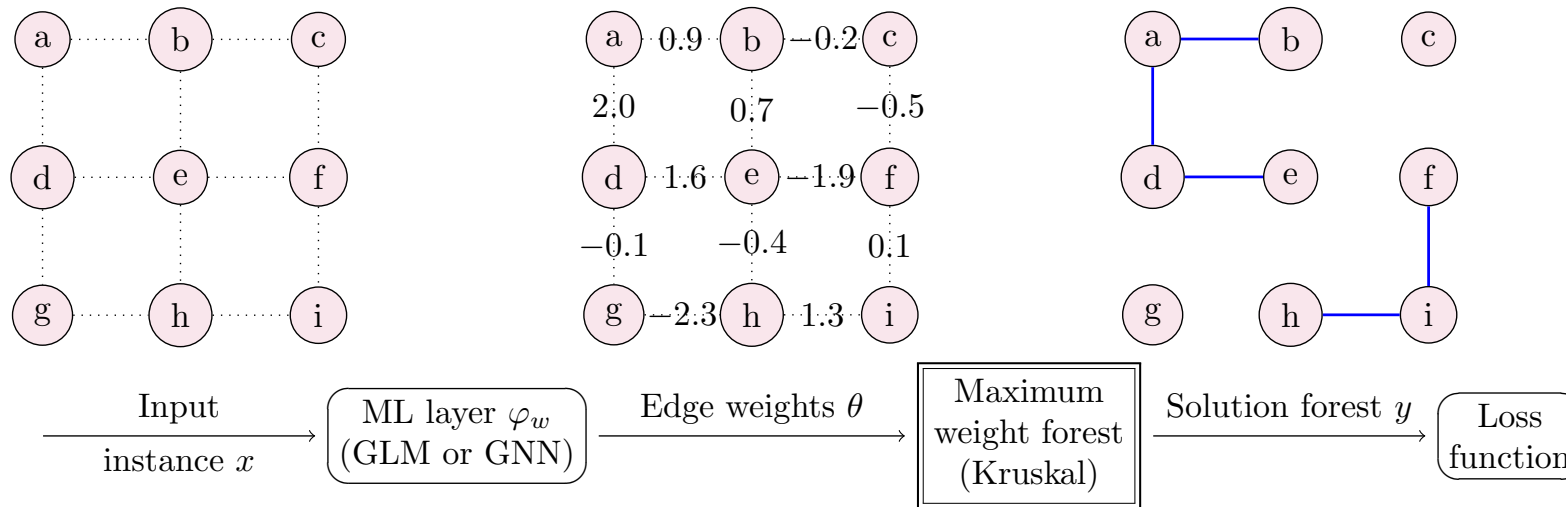
Example: stochastic vehicle scheduling

- You are: a taxi company in a city with unpredictable traffic
- I give you: a list of clients to transport
- You give me: the assignment of cars to trips



Example: 2-stage spanning trees

- You are: an electrical network operator looking to connect all vertices
- I give you: fixed design costs + variable recourse costs
- You give me: which wires to build first before the variable costs are revealed



Python libraries

- `cvxpylayers` [21]
- `TorchOpt` [22]
- `optax` [23]
- `Theseus` [24]
- `PyEP0` [25]

Julia libraries

- `DiffOpt.jl` [26]
- `ImplicitDifferentiation.jl` [27]
- `InferOpt.jl` [16]

- Deep learning relies on the combination of differentiable layers
- Each layer needs a reverse-mode chain rule for efficient gradients
- Decision-focused learning uses discrete solvers as layers
- There are various learning scenarios depending on available data
- The loss function is not necessarily differentiable
- Models must be constructed based on the problem at hand

- [1] M. Blondel and V. Roulet, “The Elements of Differentiable Programming.” [Online]. Available: <http://arxiv.org/abs/2403.14606>
- [2] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- [3] S. Scardapane, “Alice’s Adventures in a Differentiable Wonderland – Volume I, A Tour of the Land.” [Online]. Available: <http://arxiv.org/abs/2404.17625>
- [4] S. Laue, “On the Equivalence of Automatic and Symbolic Differentiation.” [Online]. Available: <http://arxiv.org/abs/1904.02990>
- [5] K. B. Petersen and M. S. Pedersen, “The Matrix Cookbook,” 2012. [Online]. Available: <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf>
- [6] M. Giles, “An Extended Collection of Matrix Derivative Results for Forward and Reverse Mode Automatic Differentiation,” 2008, [Online]. Available: <https://ora.ox.ac.uk/objects/uuid:8d0c0a29-c92b-4153-a1d2-38b276e93124>
- [7] C. C. Margossian, “A Review of Automatic Differentiation and Its Efficient Implementation,” *WIREs Data Mining and Knowledge Discovery*, vol. 9, no. 4, p. e1305, 2019, doi: [10.1002/widm.1305](https://doi.org/10.1002/widm.1305).
- [8] J. Revels, M. Lubin, and T. Papamarkou, “Forward-Mode Automatic Differentiation in Julia.” [Online]. Available: <http://arxiv.org/abs/1607.07892>
- [9] J. Bradbury *et al.*, “JAX: Composable Transformations of Python+NumPy Programs.” [Online]. Available: <http://github.com/google/jax>
- [10] G. Dalle and A. Hill, “A Common Interface for Automatic Differentiation.” [Online]. Available: <http://arxiv.org/abs/2505.05542>
- [11] J. Hückelheim, H. Menon, W. Moses, B. Christianson, P. Hovland, and L. Hascoët, “A Taxonomy of Automatic Differentiation Pitfalls,” *WIREs Data Mining and Knowledge Discovery*, vol. 14, no. 6, p. e1555, 2024, doi: [10.1002/widm.1555](https://doi.org/10.1002/widm.1555).
- [12] J. Mandi *et al.*, “Decision-Focused Learning: Foundations, State of the Art, Benchmark and Future Opportunities,” *Journal of Artificial Intelligence Research*, vol. 80, pp. 1623–1701, Aug. 2024, doi: [10.1613/jair.1.15320](https://doi.org/10.1613/jair.1.15320).

- [13] A. N. Elmachetoub and P. Grigas, “Smart ``Predict, Then Optimize”,” *Management Science*, vol. 68, no. 1, pp. 9–26, Jan. 2022, doi: [10.1287/mnsc.2020.3922](https://doi.org/10.1287/mnsc.2020.3922).
- [14] B. Wilder, B. Dilkina, and M. Tambe, “Melding the Data-Decisions Pipeline: Decision-Focused Learning for Combinatorial Optimization,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 1658–1665, July 2019, doi: [10.1609/aaai.v33i01.33011658](https://doi.org/10.1609/aaai.v33i01.33011658).
- [15] J. Mandi and T. Guns, “Interior Point Solving for LP-based Prediction+optimisation,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 7272–7282. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/51311013e51adebc3c34d2cc591fefee-Abstract.html>
- [16] G. Dalle, L. Baty, L. Bouvier, and A. Parmentier, “Learning with Combinatorial Optimization Layers: A Probabilistic Approach.” [Online]. Available: <http://arxiv.org/abs/2207.13513>
- [17] M. Niepert, P. Minervini, and L. Franceschi, “Implicit MLE: Backpropagating Through Discrete Exponential Family Distributions,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2021, pp. 14567–14579. [Online]. Available: <https://proceedings.neurips.cc/paper/2021/hash/7a430339c10c642c4b2251756fd1b484-Abstract.html>
- [18] M. Paulus, D. Choi, D. Tarlow, A. Krause, and C. J. Maddison, “Gradient Estimation with Stochastic Softmax Tricks,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 5691–5704. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/3df80af53dce8435cf9ad6c3e7a403fd-Abstract.html>
- [19] M. Blondel, A. F. T. Martins, and V. Niculae, “Learning with Fenchel-Young Losses,” *Journal of Machine Learning Research*, vol. 21, no. 35, pp. 1–69, 2020, [Online]. Available: <http://jmlr.org/papers/v21/19-021.html>
- [20] P. Veličković, “Message Passing All the Way Up.” [Online]. Available: <http://arxiv.org/abs/2202.11097>
- [21] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, “Differentiable Convex Optimization Layers,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2019. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/9ce3c52fc54362e22053399d3181c638-Abstract.html>

- [22] J. Ren *et al.*, “TorchOpt: An Efficient Library for Differentiable Optimization,” *Journal of Machine Learning Research*, vol. 24, no. 367, pp. 1–14, 2023, [Online]. Available: <http://jmlr.org/papers/v24/23-0191.html>
- [23] DeepMind *et al.*, “The DeepMind JAX Ecosystem.” [Online]. Available: <http://github.com/google-deepmind>
- [24] L. Pineda *et al.*, “Theseus: A Library for Differentiable Nonlinear Optimization.” [Online]. Available: <http://arxiv.org/abs/2207.09442>
- [25] B. Tang and E. B. Khalil, “PyEPO: A PyTorch-based End-to-End Predict-Then-Optimize Library for Linear and Integer Programming,” *Mathematical Programming Computation*, vol. 16, no. 3, pp. 297–335, Sept. 2024, doi: [10.1007/s12532-024-00255-x](https://doi.org/10.1007/s12532-024-00255-x).
- [26] A. Sharma, M. Besançon, J. D. Garcia, and B. Legat, “Flexible Differentiable Optimization via Model Transformations.” [Online]. Available: <http://arxiv.org/abs/2206.06135>
- [27] G. Dalle, “Machine Learning and Combinatorial Optimization Algorithms, with Applications to Railway Planning,” Doctoral dissertation, 2022. [Online]. Available: <https://pastel.hal.science/tel-04053322>